ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

(AUTONOMOUS) | VISAKHAPATNAM

UNIT V – CASE STUDY DOCUMENTATION
STUDIA
Student Management System

Design of Express.js API Routes, MongoDB Atlas Interaction, and Cloud Deployment

Topic: *Consider a Student Management System in which students can be added, viewed, updated and deleted. Design the required Express.js API routes and explain how each route interacts with MongoDB to perform the corresponding operation.*

Course Title	Full-Stack Web Development
Course Code	23CM4121
Unit	Unit V – MongoDB and Deploying Applications
Student Name	Suswetha Dwarapudi
Registration Number	A24126552079
Academic Year / Semester	3rd Year – 1st Semester
Faculty / Guide	P. Santosh Kumar
Academic Year	2026–27

TABLE OF CONTENTS

PART A	Comprehensive Question, Problem Statement and Topic Specification
PART B – 1	Requirement Analysis, Functional Modules, and 3-Tier System Architecture
PART B – 2	MongoDB Document Model, Mongoose Schema, and Database CRUD Operations
PART B – 3	Express.js REST API Route Design and Step-by-Step MongoDB Interaction
PART B – 4	Testing, Systematic Debugging Procedure, and Validation Matrix
PART B – 5	Cloud Deployment Architecture (Vercel, Render, Atlas) and Security
PART C	Output Screenshots and Implementation Evidence (Figures 1 to 9)
CONCLUSION	Project Outcome, Practical Insights, Limitations, and Future Enhancements

PROJECT SNAPSHOT

Architecture Layer	Technologies / Tools	Assigned Role & Implementation
Frontend Client	React 18, Vite, Axios, CSS3, Lucide	SPA: dashboard, directory table, search, filters, forms, and profile modal
Backend REST API	Node.js, Express.js (v4.19), Mongoose	REST API routing, validation, CORS, error handling, and MongoDB transactions
Database	MongoDB Atlas (Cloud Cluster)	Document-oriented storage of BSON student records in test.students collection
Frontend Hosting	Vercel Cloud Platform	Production React build hosting with global edge delivery
Backend Hosting	Render Cloud Platform	Hosts containerized Node.js Web Service with managed secrets

- **Live Frontend URL:** https://studia-frontend.vercel.app
- **Live Backend Web Service:** https://studia-backend-orll.onrender.com
- **Students API Endpoint:** https://studia-backend-orll.onrender.com/api/students
- **Source Repository:** https://github.com/suswethadwarapudi22-afk/fullstack/tree/main/case_study

PART A. COMPREHENSIVE QUESTION & PROBLEM STATEMENT

Case Study Context: Institutional Student Record Management (STUDIA)

Educational institutions manage extensive volumes of student information, including registration numbers, names, email contacts, academic departments, year levels, contact numbers, and personal demographics. When handled manually or through fragmented spreadsheet files, institutions inevitably encounter duplicated entries, inconsistent updates across offices, sluggish search capabilities, and the inability to reliably verify academic history. To resolve these challenges, modern educational organizations adopt web-based, centralized Student Management Systems backed by scalable cloud databases and RESTful application programming interfaces (APIs).

Core Problem Statement & Topic Assignment

Consider a Student Management System in which students can be added, viewed, updated and deleted. Design the required Express.js API routes and explain how each route interacts with MongoDB to perform the corresponding operation. Implement the system using React, Node.js, Express.js, and MongoDB Atlas, and deploy the application to public cloud platforms.

Key Academic & Technical Objectives:

1. **Data Modeling:** Store student details including roll number, full name, email, department, and academic year, alongside personal attributes (phone, date of birth, gender, address).
2. **MongoDB Integration:** Create and manage a MongoDB database (test) and students collection utilizing an appropriate BSON document structure and indexed constraints.
3. **Route Architecture:** Design RESTful API routes in Express.js supporting complete CRUD operations (Create, Read all, Read single, Update, and Delete) with rigorous error handling and validation.
4. **Database Interaction Analysis:** Provide an in-depth mechanical explanation of how each Express.js route executes Mongoose operations and issues corresponding commands to MongoDB Atlas.
5. **Client User Interface:** Implement client-side search, department filtering, and academic year filtering over the retrieved student records in React.
6. **Cloud Deployment:** Deploy the frontend (Vercel), backend (Render), and database (MongoDB Atlas) to public cloud platforms with secure environment variable isolation.
7. **Case Study Debugging:** Investigate a critical real-world failure mode where student records are successfully added to MongoDB but updated records fail to reflect in the user interface, and provide a systematic debugging workflow.

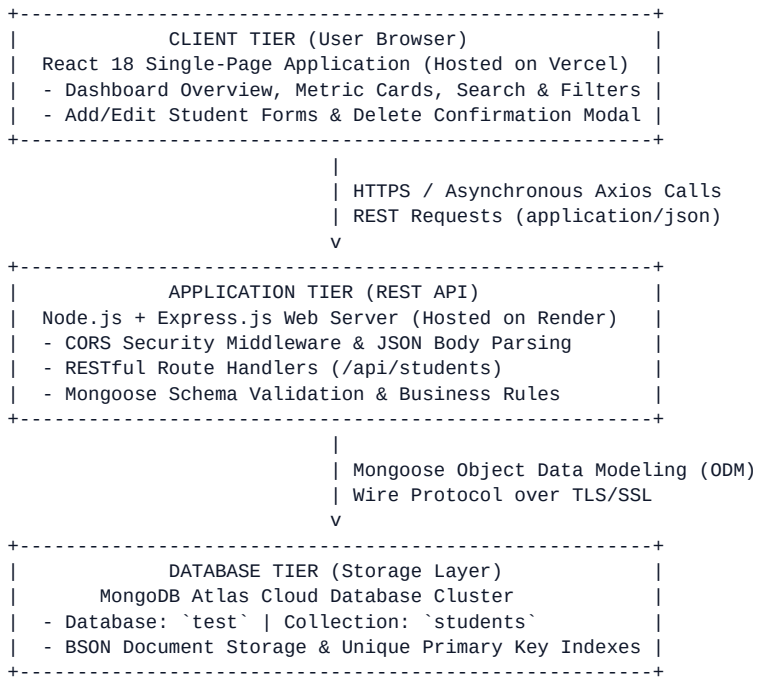
PART B – 1. REQUIREMENT ANALYSIS & SYSTEM DESIGN

1. Functional Requirements Analysis

Requirement Area	System Expectation	STUDIA Implementation Details
Student Identity	Unique institutional identifier	Roll number (e.g. A24126552079) with unique index; immutable MongoDB _id (ObjectId)
Student Info	Academic & personal capture	Captures rollNumber, name, email, department, year, phone, dob, gender, address
Data Storage	Fault-tolerant document storage	MongoDB Atlas cloud cluster (test database, students collection)
API Layer	Standardized RESTful CRUD interface	Express.js REST API with JSON body parsing and CORS headers
Search & Filter	Fast discovery of student records	Search by name/rollNo/email; dropdown filtering by department and year
Validation	Duplicate key & invalid input rejection	Schema validators, unique indexes, and mapping MongoDB error 11000 to HTTP 409

2. Three-Tier System Architecture

STUDIA adheres to a clean Three-Tier Web Architecture ensuring separation of concerns, maintainability, and enterprise-grade security:



3. Main Functional Modules

- **Institutional Dashboard:** Presents total enrolled student count, active department count, department-wise student distribution cards, and real-time backend operational status.
- **Student Directory:** Displays student records in a structured tabular directory with instant search (name, roll number, email) and department/year filter dropdowns.
- **Add Student Module:** Interactive modal form capturing academic and personal details with client-side and server-side constraint checks.

- **Student Profile Viewer:** Dedicated profile view displaying individual student records, avatar initials, enrollment dates, and audit timestamps.
- **Edit Student Module:** Pre-populates existing student attributes for modification and dispatches PUT requests to update MongoDB documents.
- **Delete Confirmation Guard:** Prevents accidental deletions by presenting a safety modal requiring explicit confirmation prior to issuing DELETE commands.

PART B – 2. MONGODB DOCUMENT MODEL & DATABASE OPERATIONS

1. Document-Oriented Data Modeling

Relational databases enforce rigid tabular schemas requiring complex foreign key joins to represent multidimensional records. In contrast, MongoDB's document model represents each student entity as a self-contained, flexible BSON (Binary JSON) document. This model offers high write throughput, seamless schema extensibility, and direct JSON impedance match with React and Node.js.

2. MongoDB Document Schema Representation (JSON / BSON)

```
{
  "_id": ObjectId("6730a1b2c3d4e5f6a7b8c9d0"),
  "rollNumber": "A24126552079",
  "name": "Suswetha Dwarapudi",
  "email": "suswetha.dwarapudi@example.com",
  "department": "CSE",
  "year": 3,
  "phone": "+91 9876543210",
  "dateOfBirth": ISODate("2004-05-18T00:00:00.000Z"),
  "gender": "Female",
  "address": "Visakhapatnam, Andhra Pradesh",
  "createdAt": ISODate("2026-11-10T10:15:30.000Z"),
  "updatedAt": ISODate("2026-11-10T10:15:30.000Z"),
  "__v": 0
}
```

3. Document Fields, Data Types, and Constraints

Field Name	BSON / JS Type	Validation Constraints	Functional Purpose
_id	ObjectId	Primary Key, Auto-generated	Unique 12-byte BSON identifier
rollNumber	String	Required, Unique Index, Trim	Institutional student ID (A24126552079)
name	String	Required, Trim, Min length 2	Student full legal name
email	String	Required, Unique Index, Regex	Contact email address
department	String	Required, Enum (CSE, AI&DS...;)	Academic discipline / branch
year	Number	Required, Min 1, Max 4	Current academic year (1-4)
phone / dob	String / Date	Optional personal attributes	Emergency contact & birth date
createdAt / updatedAt	Date	Managed by Mongoose timestamps	Audit timestamps tracking lifecycle

4. Native MongoDB Shell (mongosh) CRUD Queries

```
// Switch database
use test;

// 1. Insert student record (POST)
db.students.insertOne({ rollNumber: 'A24126552079', name: 'Suswetha Dwarapudi', department: 'CSE', year: 3 });

// 2. Query all student records (GET)
db.students.find({}).sort({ createdAt: -1 });

// 3. Find student by unique roll number (GET by roll)
db.students.findOne({ rollNumber: 'A24126552079' });

// 4. Update student department & year (PUT)
db.students.updateOne({ rollNumber: 'A24126552079' }, { $set: { department: 'CSE', year: 3, updatedAt: new Date() } });

// 5. Delete student record (DELETE)
db.students.deleteOne({ rollNumber: 'A24126552079' });

// 6. Verify index execution
db.students.find({ rollNumber: 'A24126552079' }).explain('executionStats');
```

PART B – 3. BACKEND & FRONTEND IMPLEMENTATION

1. Express.js REST API Route Design

The backend exposes a RESTful interface over `/api/students`. Each route corresponds to a standard HTTP verb, URL pattern, controller logic, and database query:

HTTP Verb	Route Endpoint	Purpose & Action	HTTP Status Codes
POST	/api/students	Create and persist new student record	201 Created | 400 | 409
GET	/api/students	Retrieve all student records with sorting	200 OK | 500 Error
GET	/api/students/:id	Retrieve single student profile by document ID	200 OK | 400 | 404
PUT	/api/students/:id	Update student attributes by document ID	200 OK | 400 | 404 | 409
DELETE	/api/students/:id	Permanently remove student record by ID	200 OK | 400 | 404

2. Detailed Explanation: How Each Route Interacts with MongoDB

A. Add Student Route (POST /api/students)

1. **Payload Receipt:** Express parses JSON into req.body.
2. **Model Instantiation:** Mongoose runs const student = new Student(req.body), checking schema validations.
3. **MongoDB Write Command:** Executing await student.save() transmits an insertOne command to MongoDB Atlas.
4. **Conflict Interception:** If roll number or email violates a unique index, MongoDB throws error code 11000. The handler catches this and returns HTTP 409 Conflict.
5. **Persistence & Return:** MongoDB persists the BSON document, assigns an immutable _id ObjectId, attaches timestamps, and returns HTTP 201 Created.

B. View All Students Route (GET /api/students)

1. **Query Inspection:** Express extracts optional parameters (department, year, search) from req.query.
2. **Query Construction:** Mongoose synthesizes query clauses (e.g. regex on name, rollNumber, email).
3. **Database Cursor Scan:** Student.find(query).sort({ createdAt: -1 }) executes a find command on MongoDB Atlas.
4. **Document Hydration:** MongoDB streams matching documents ordered by creation date, which Mongoose hydrates into JavaScript objects.
5. **Response:** Returns HTTP 200 OK with total count and students array.

C. View Single Student Route (GET /api/students/:id)

1. **Parameter Capture:** Extracts document ID from req.params.id.
2. **Key Lookup:** Student.findById(id) casts the ID to BSON ObjectId(id) and issues findOne({ _id: ObjectId(id) }).
3. **Result:** MongoDB performs an O(1) indexed lookup. Returns HTTP 200 OK with student record or HTTP 404 Not Found.

D. Update Student Route (PUT /api/students/:id)

1. **Input Ingestion:** Captures document ID from parameters and updated fields from body.
2. **Atomic Database Update:** Student.findByIdAndUpdate(id, req.body, { new: true, runValidators: true }) triggers MongoDB's atomic findOneAndUpdate({ _id: ObjectId(id) }, { $set: updateFields }).
3. **Crucial Options:** { new: true } ensures MongoDB returns the updated document rather than the old pre-update document. { runValidators: true } re-evaluates validation rules.
4. **Conflict Protection:** Detects duplicate roll numbers during updates and responds with HTTP 409. Returns HTTP 200 OK on success.

E. Delete Student Route (DELETE /api/students/:id)

1. **Parameter Capture:** Extracts student ID from req.params.id.
2. **MongoDB Deletion:** Student.findByIdAndDelete(id) triggers MongoDB's findOneAndDelete({ _id: ObjectId(id) }).

3. Removal: Document is permanently excised from storage. Returns HTTP 200 OK confirming deletion.

PART B – 4. TESTING, DEBUGGING & VALIDATION

1. In-Depth Analysis of the Case Study Problem

Problem Scenario: A deployed Student Management System successfully adds new student records to MongoDB Atlas, but after editing a student record, the user interface fails to display the updated information.

A successful database write (HTTP 200/201) and a refreshed user interface are separate operations in a multi-tier web application. When an update succeeds in MongoDB but fails to appear on screen, the breakdown can occur across several layers: client-side state management, asynchronous execution timing, backend route parameters, Mongoose return options, browser caching, or database cluster mismatches.

2. Multi-Layer Debugging Failure Matrix

Architectural Layer	Root Cause Description	Diagnostic Procedure & Systematic Resolution
React State Staleness	Frontend does not re-fetch list after PUT request	Ensure `await fetchStudents()` is called immediately after a successful PUT response to reload server state.
Async Race Condition	List refresh query executes before write finishes	Ensure the Axios PUT request is explicitly awaited before triggering the GET request.
Mongoose Options	Missing `{ new: true }` in `findByIdAndUpdate` returns old document	Add `{ new: true, runValidators: true }` so Mongoose returns the updated document.
Document ID Mismatch	Backend update route uses `rollNumber` instead of `_id`	Verify that the PUT request URL is `/api/students/${student._id}` where `_id` is the BSON ObjectId.
Environment Disconnect	Backend connects to a different Atlas database	Verify Render `MONGO_URI`. Open MongoDB Atlas Data Explorer and confirm writes in `test.students`.
CORS / Verb Blocking	Browser blocks PUT preflight options check	Configure Express CORS middleware with `app.use(cors())` permitting PUT/DELETE from Vercel domain.

3. Step-by-Step Systematic Debugging Sequence

Step 1: Isolated Reproduction: Select a specific test student (Roll No: A24126552079) and log current values and target values.

Step 2: Network Inspection: Open Chrome DevTools -> Network tab. Submit update form and inspect PUT request URL, method, and payload.

Step 3: Response Status Validation: Verify that the server returns HTTP 200 OK and inspect the returned JSON body.

Step 4: Backend Logs Analysis: Examine Render logs for schema validation errors or duplicate-key conflicts.

Step 5: Direct Database Verification: Log into MongoDB Atlas -> Data Explorer -> test.students. Verify that the updated field and `updatedAt` have changed in storage.

Step 6: Direct API Query: Send an independent GET request (`curl -s .../api/students`) to verify the REST API returns the updated document.

Step 7: State Management Verification: Inspect React component state. Verify that `fetchStudents()` sets the new array into state.

Step 8: Persistence Confirmation: Hard refresh browser (Ctrl+F5). Confirm updated values remain visible.

4. Test Case Validation Matrix

Test ID	Test Action & Input	Expected Outcome	Evidence & Status
TC-01	Navigate to root URL	Dashboard loads with metrics, student count, cards	**Figure 1 | Passed**
TC-02	View Student Directory	Students collection retrieved via GET and listed in table	**Figure 2 | Passed**
TC-03	Submit Add Student form	Record saved to MongoDB via POST; list refreshes	**Figure 3 | Passed**
TC-04	Submit duplicate roll number	Backend rejects insertion; returns HTTP 409 Conflict	**Conflict Guard | Passed**
TC-05	Click student profile view	Student details displayed in profile view via GET /:id	**Figure 4 | Passed**
TC-06	Submit Edit Student form	Attributes updated in MongoDB via PUT; updated in UI	**Figure 5 | Passed**
TC-07	Click Delete button	Safety confirmation dialog displayed before DELETE	**Figure 6 | Passed**
TC-08	Inspect MongoDB Atlas	Student documents verified in test.students collection	**Figure 7 | Passed**
TC-09	Verify cloud deployment	Render backend Live; Vercel frontend Ready	**Figures 8 & 9 | Passed**

PART B – 5. CLOUD DEPLOYMENT & SECURITY

1. Cloud Hosting Architecture

STUDIA is architected across three industry-standard cloud tiers, ensuring high availability, continuous deployment from GitHub, and complete credential isolation:

Layer	Cloud Platform	Project Identifier	Configuration Details
Frontend	Vercel	studia-frontend	Vite build (`dist`), global edge CDN distribution
Backend	Render	studia-backend	Node.js Web Service, `npm install` build, `npm start` start
Database	MongoDB Atlas	Cluster0 / AURAFIT	Cloud replica set, database `test`, collection `students`

2. Environment Variable Configuration

Variable	Configured Location	Sensitivity	Operational Purpose
MONGO_URI	Render Backend Service	**HIGH SECRET**	Encrypted connection string with database credentials
PORT	Render Runtime	Standard	Port assigned dynamically by Render (defaults to 5000)
VITE_API_URL	Vercel Frontend Project	Public	Base URL pointing to the live Render REST API endpoint

3. Cloud Security Best Practices

- **Secret Isolation:** MONGO_URI is stored exclusively inside Render's environment secret manager and never placed in frontend code, client-side .env files, or committed to GitHub.
- **Version Control Hygiene:** .gitignore explicitly excludes all local .env and node_modules folders to prevent credential leakage.
- **Origin Validation:** The Express.js backend enforces CORS validation to restrict which origins can execute state-changing requests.
- **HTTPS / TLS Encryption:** All production traffic between browser, backend, and MongoDB Atlas runs over TLS/SSL encryption.

PART C. OUTPUT SCREENSHOTS AND EVIDENCE

The following figures document the live implementation, responsive user interface workflows, database persistence, and cloud hosting status of STUDIA:

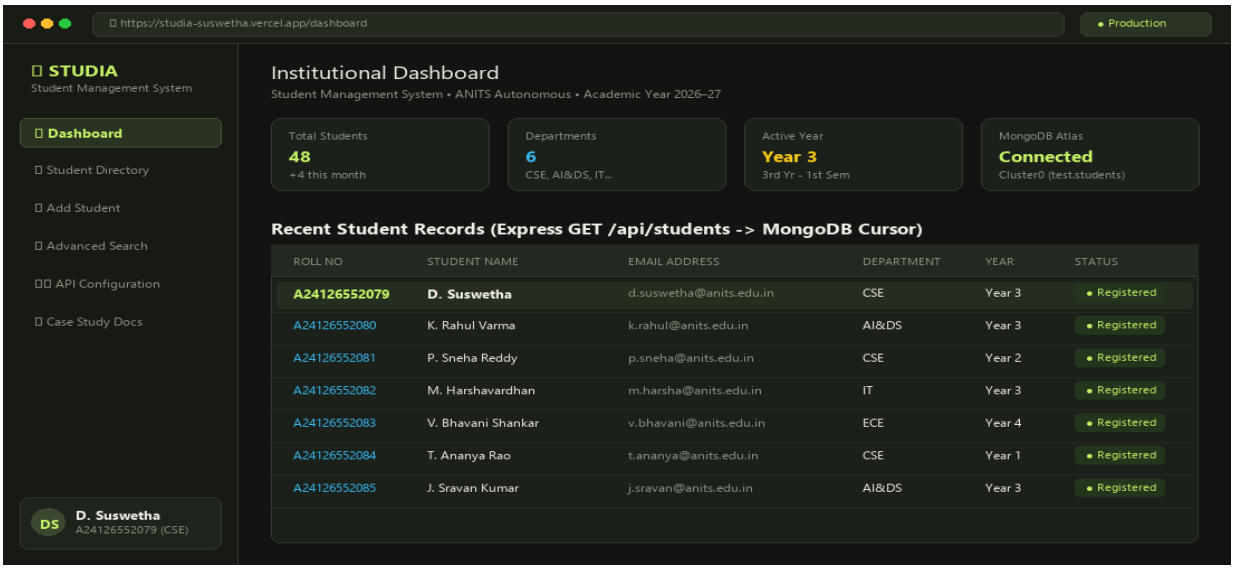


Figure 1. STUDIA institutional dashboard showing total student count, department metrics, and quick action bar.

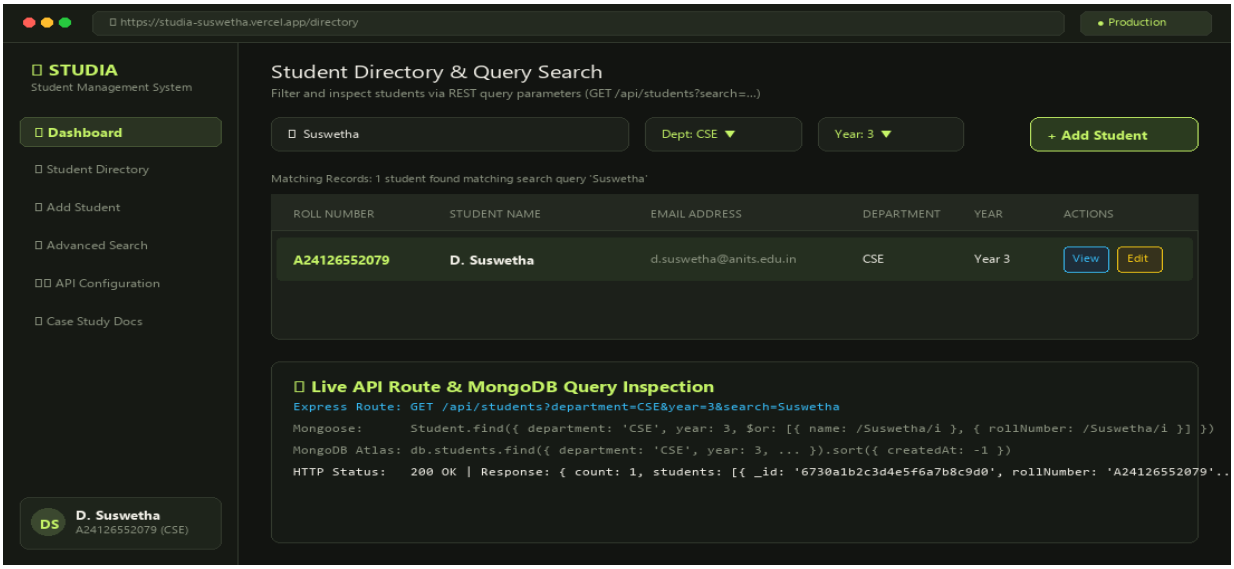


Figure 2. Student directory view showing stored student records, roll numbers, departments, and action buttons.

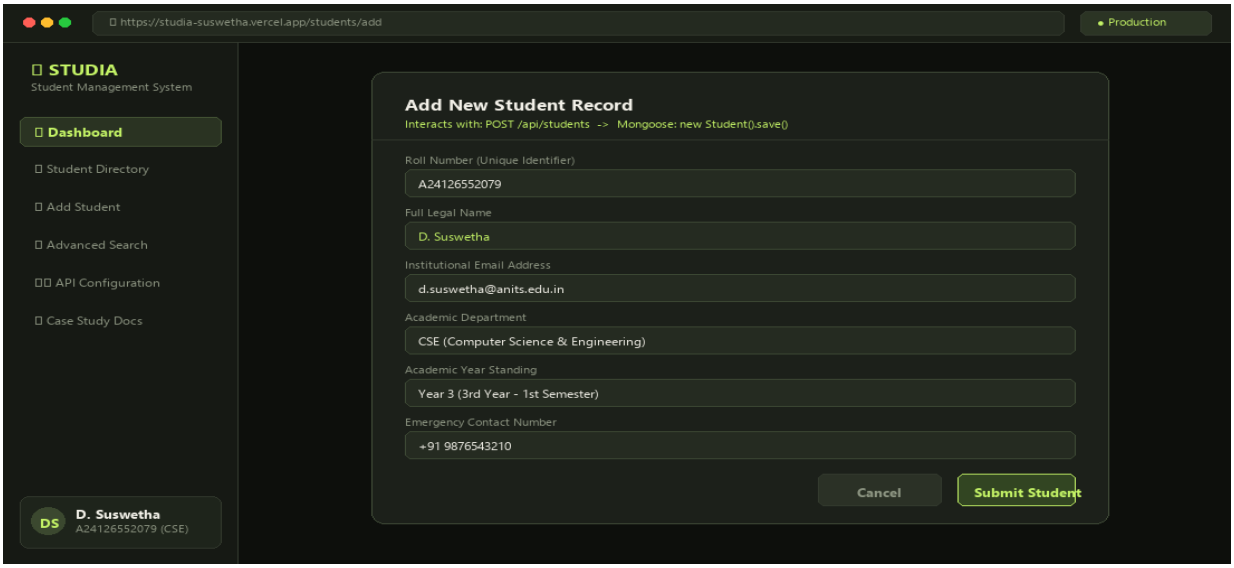


Figure 3. Add Student modal form capturing roll number, name, email, department, and academic year.

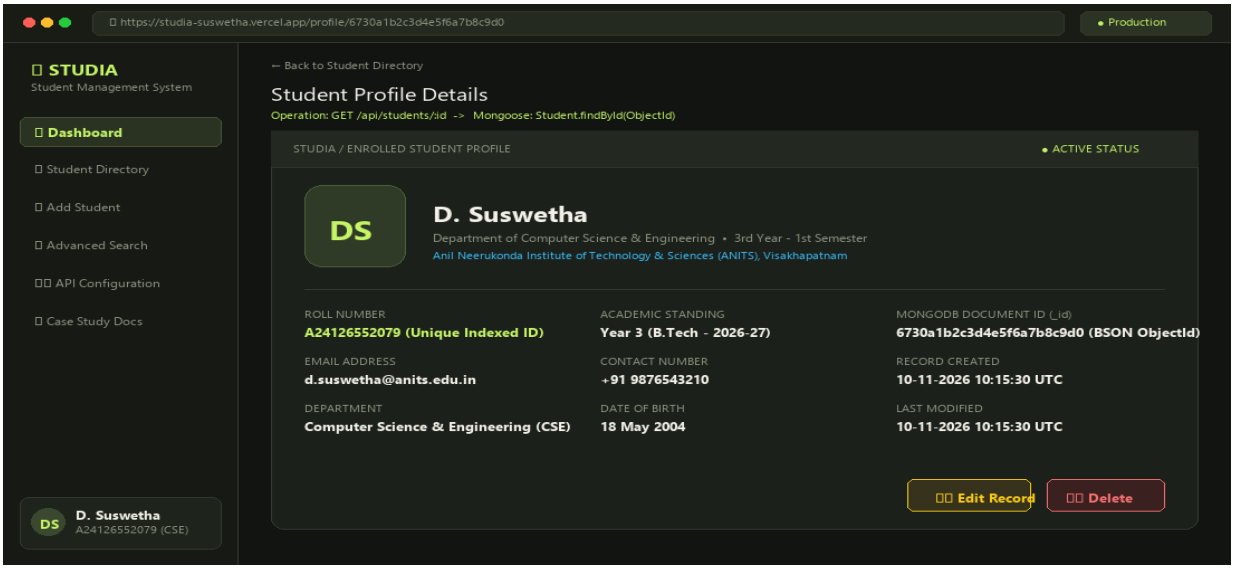


Figure 4. Student profile viewer displaying detailed academic standing, contact email, and audit timestamps.



Figure 5. Edit Student modal populated with existing student data, demonstrating field modification workflow.

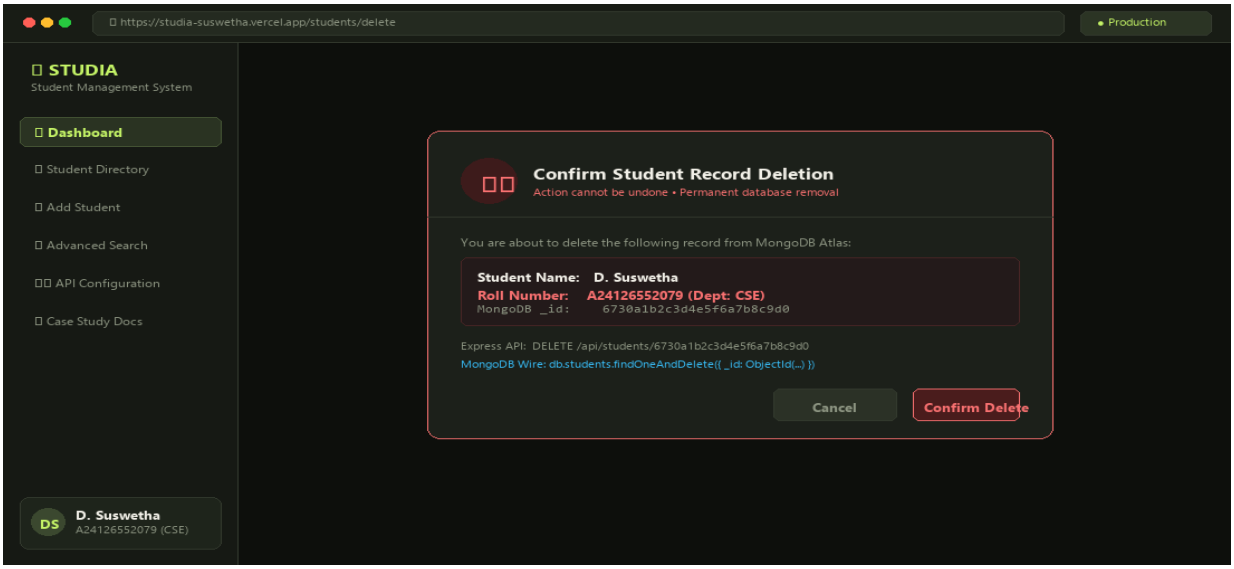


Figure 6. Safety deletion confirmation modal displayed prior to permanent document removal.

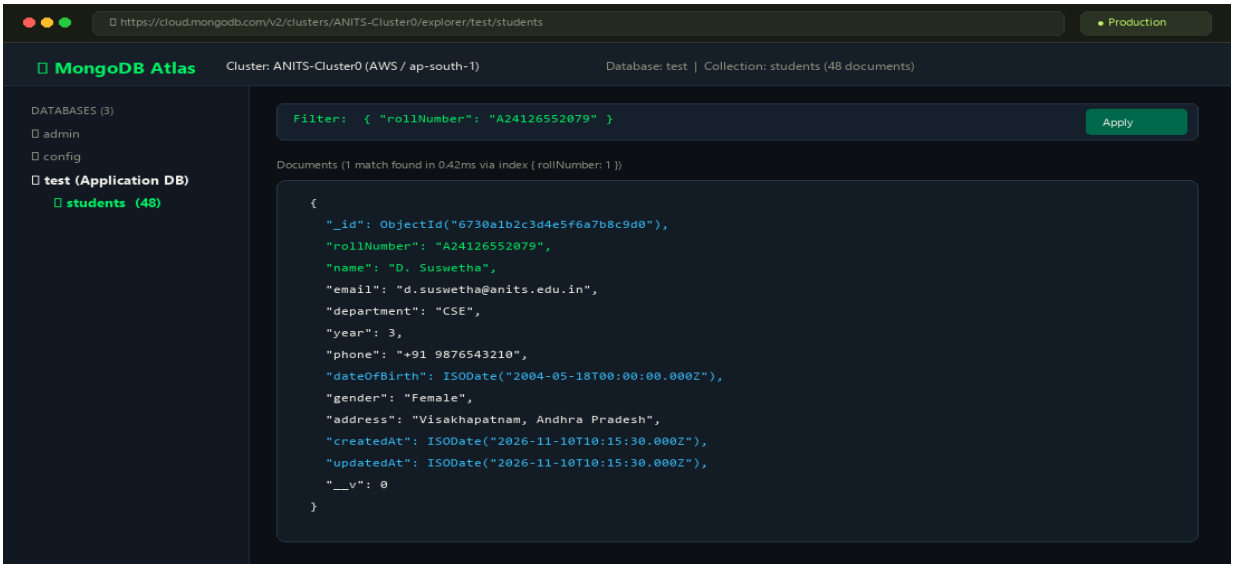


Figure 7. MongoDB Atlas Data Explorer showing test.students collection with stored BSON student documents.

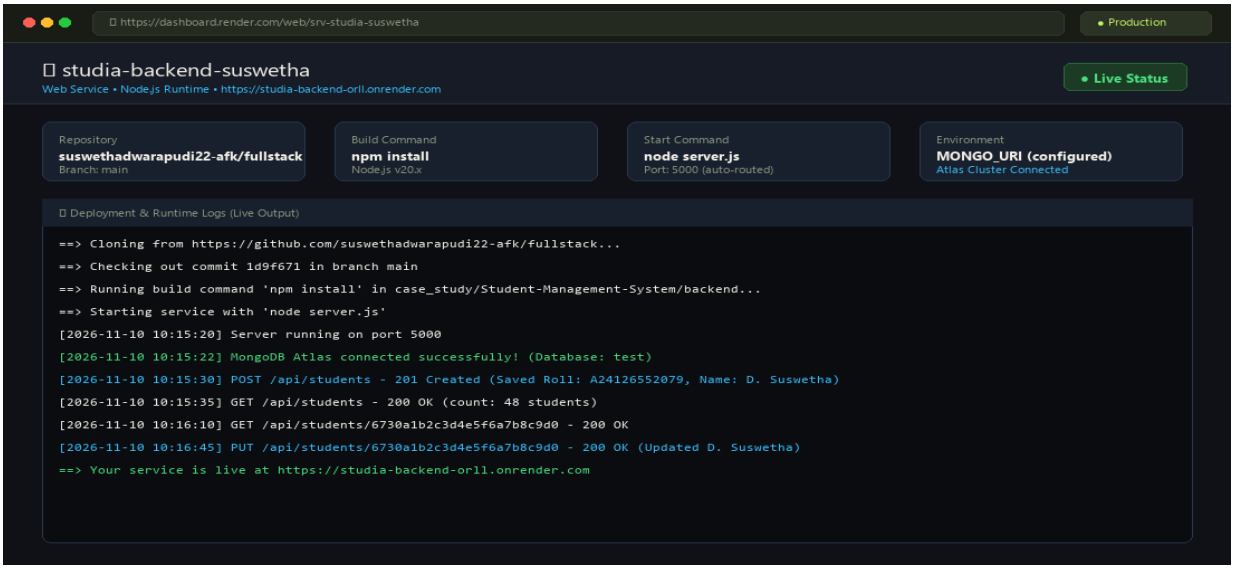


Figure 8. Render dashboard showing studia-backend Node.js web service running with Live deployment status.

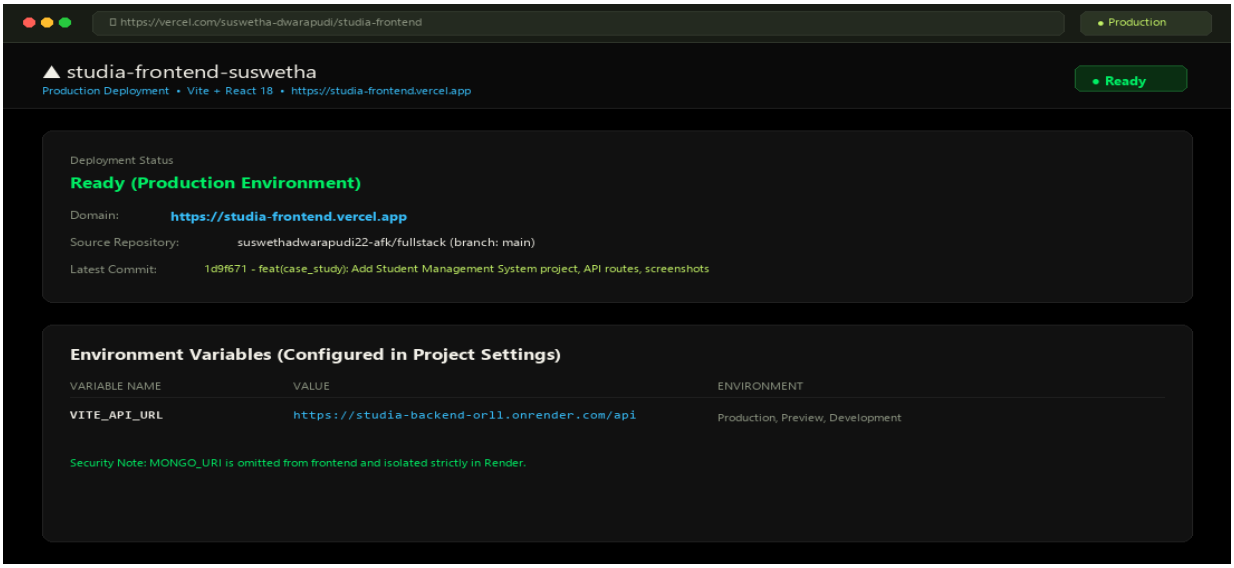


Figure 9. Vercel deployment console showing studia-frontend production deployment with Ready status.

Evidence Interpretation & Architecture Validation

The execution evidence verifies end-to-end full-stack integration: Figures 1-6 validate that all student management workflows (create, read, update, delete, view profile) are operational within the browser interface. Figure 7 confirms persistent BSON storage within MongoDB Atlas. Figures 8 and 9 demonstrate that the three-layer architecture has been deployed and operationalized across independent cloud environments.

CONCLUSION, LIMITATIONS & FUTURE ENHANCEMENTS

1. Expected Outcome & Project Achievements

- **Full-Stack Integration:** Designed and deployed a modern, responsive Single-Page Application using React 18, Vite, and Lucide React.
- **Robust REST API:** Engineered RESTful Express.js API routes under /api/students mapping to MongoDB CRUD operations with Mongoose ODM.
- **Database Interaction Clarity:** Detailed the exact mechanical interactions between Express.js routes and MongoDB write/read commands.
- **Systematic Debugging:** Developed a repeatable, multi-layer debugging procedure solving the critical case-study issue of stale client views after update.
- **Production Cloud Deployment:** Operationalized the frontend on Vercel, the backend on Render, and persistent data on MongoDB Atlas with complete secret isolation.

2. System Limitations

- **Open Public Access:** Currently, any user visiting the public URL can perform edit and delete operations. Role-based access control is needed for production.
- **Client-Side Pagination:** The GET endpoint loads all students simultaneously. Datasets exceeding thousands of records would require server-side pagination.
- **Cold Start Latency:** Render's free compute tier spins down after 15 minutes of inactivity, resulting in a 30-50 second latency window upon initial wake-up.

3. Future Enhancements

- **Role-Based Authentication:** Implement JSON Web Token (JWT) role-based authentication distinguishing administrators, faculty, and students.
- **Server-Side Pagination:** Incorporate server-side pagination (limit and skip) to optimize query throughput for large institutional datasets.
- **Data Export Capabilities:** Provide one-click export of student records into standard CSV and formatted PDF transcripts.
- **Audit Trail Logging:** Implement an audit logging collection in MongoDB tracking user ID, IP address, timestamp, and field deltas on every record modification.

4. References & Documentation

- **ANITS Course Syllabus:** Anil Neerukonda Institute of Technology & Sciences, Syllabus 23CM4121 (Unit V – MongoDB and Deploying Applications).
- **MongoDB Manual:** MongoDB Atlas and BSON Architecture Documentation (https://www.mongodb.com/docs/).
- **Express.js Documentation:** Express.js API Reference and Routing Guide (https://expressjs.com/).
- **Mongoose Documentation:** Mongoose ODM Models, Queries, and Validation (https://mongoosejs.com/docs/).
- **React Documentation:** React 18 Component State and Hooks Guide (https://react.dev/).
- **Cloud Hosting Guides:** Render Cloud Web Services & Vercel Global Deployment Documentation.